

LAPORAN

Tugas Praktik Demonstrasi

FR.IA.02 — Kelompok Pekerjaan 1 & 2

SIPERPUS

Sistem Informasi Peminjaman Buku

Skema Sertifikasi	Pemrogram Junior (Junior Coder)
Jenis Skema	Okupasi — Level KKNi 2
Nomor Skema	16-05/SSO/020502/X/2021
Tempat Uji Kompetensi	POLINELA, Bandar Lampung
Lembaga	LSP Entrepreneur Digital Indonesia
Nama Asesi	Irfana Dwi Fangga
Tanggal	26 Agustus 2026
Repository	https://github.com/irfanadwifangga/sertifikasi-lspedi

DOKUMENTASI KODE PROGRAM

Tugas Praktik Demonstrasi (TPD) Kelompok Pekerjaan 2 — Debugging & Dokumentasi Program

Nama Asesi	Irfana Dwi Fangga
Nama Aplikasi	SIPERPUS — Sistem Informasi Peminjaman Buku
Skema Sertifikasi	Pemrogram Junior (Junior Coder)
Nomor Skema	16-05/SSO/020502/X/2021
Jenis Skema	Okupasi — Level KKNi 2
Lembaga Sertifikasi	LSP Entrepreneur Digital Indonesia
Tempat Uji Kompetensi	POLINELA, Bandar Lampung
Tanggal	26 Agustus 2026
Repository	https://github.com/irfanadwifangga/sertifikasi-lspedi

1. Ringkasan Aplikasi

SIPERPUS adalah aplikasi web pengelolaan peminjaman buku perpustakaan yang dibangun memakai framework **NestJS** dengan bahasa **TypeScript**. Aplikasi terdiri atas **8 halaman** (melebihi syarat minimal 5 halaman) dan menyimpan datanya pada basis data **PostgreSQL** melalui ORM **TypeORM**.

Aplikasi mengelola dua entitas yang saling berelasi — **Buku** dan **Peminjaman** — sehingga operasi basis datanya mencakup relasi antar tabel (JOIN), kunci asing (foreign key), serta transaksi ACID untuk menjaga konsistensi stok buku.

Fitur utama:

- Dashboard berisi kartu statistik dan dua grafik interaktif (Chart.js)
- Manajemen data buku lengkap (Create, Read, Update, Delete) dengan pencarian
- Pencatatan transaksi peminjaman dengan pengurangan stok otomatis
- Pemrosesan pengembalian buku disertai **perhitungan denda keterlambatan**
- Pengubahan transaksi: koreksi identitas, perpanjangan jatuh tempo, dan **perubahan status peminjaman** yang ikut menyesuaikan stok buku
- Pencetakan **Laporan Peminjaman dalam format PDF** per periode
- Pengiriman **notifikasi surel** (bukti peminjaman & pengingat jatuh tempo)

Seluruh sistem dijalankan melalui **Docker Compose** yang terdiri atas empat container: web server Nginx, aplikasi NestJS, database server PostgreSQL, dan server surel Mailpit.

2. Hasil Debugging dan Pengujian

2.1 Bug yang ditemukan dan diperbaiki selama pengembangan

No	Gejala	Penyebab	Perbaikan
1	Aplikasi gagal menyala dengan error <code>hbs.registerPartials is not a function</code>	Modul <code>hbs</code> bergaya CommonJS diimpor sebagai namespace (<code>import * as hbs</code>), sehingga TypeScript membungkusnya dengan <code>__importStar</code> dan method aslinya tidak terekspos	Diubah menjadi default import (<code>import hbs from 'hbs'</code>) pada <code>main.ts</code> dan <code>hbs-helpers.ts</code>

No	Gejala	Penyebab	Perbaikan
2	Kolom Kode dan tanggal pada laporan PDF terpotong menjadi dua baris	Lebar kolom tabel PDF lebih sempit daripada teks terpanjang (PJM-20260811-001, 1 September 2026)	Lebar kolom disesuaikan dan ukuran huruf tabel diturunkan menjadi 8pt pada <code>pdf.service.ts</code>
3	Objek <code>Date</code> acuan penomoran kode transaksi ikut termutasi	Pemanggilan <code>setHours()</code> mengubah objek tanggal aslinya	Ditambahkan fungsi murni <code>awalHari()</code> dan <code>akhirHari()</code> yang selalu mengembalikan objek <code>Date</code> baru
4	Kedua grafik dashboard mati setelah berkas diformat ulang	Prettier memperlakukan isi <code><script></code> di dalam berkas <code>.hbs</code> sebagai teks biasa dan mengalirkannya seperti paragraf, sehingga tidak lagi berupa JavaScript valid	Skrip dipindahkan ke berkas <code>public/js/dashboard.js</code> ; data dikirim lewat atribut <code>data-*</code> pada elemen <code><canvas></code>
5	Pesan error PostgreSQL berbahasa Inggris bocor ke layar pengguna saat menghapus buku	Penjaga penghapusan hanya memeriksa buku yang <i>sedang</i> dipinjam, padahal batasan <code>ON DELETE RESTRICT</code> juga menolak buku yang punya riwayat transaksi selesai	Ditambahkan pemeriksaan lapis kedua atas jumlah riwayat transaksi pada <code>book.service.ts</code> , dengan pesan berbahasa Indonesia
6	Error kompilasi <code>Property 'stack' does not exist on type 'unknown'</code>	Variabel pada blok <code>catch</code> bertipe <code>unknown</code> ; mengakses <code>.message/.stack</code> secara langsung tidak aman karena JavaScript mengizinkan <code>throw</code> atas nilai apa pun	Dibuat fungsi bantu <code>errorMessage()</code> dan <code>errorStack()</code> di <code>common/error.ts</code> ; opsi <code>useUnknownInCatchVariables</code> diaktifkan agar kesalahan sejenis tertangkap saat build
7	Setiap permintaan ke alamat yang tidak ada (termasuk <code>/favicon.ico</code>) tercatat di log sebagai ERROR beserta stack trace penuh	Penangkap error memperlakukan seluruh status secara sama, padahal 4xx adalah kesalahan sisi permintaan dan bukan kegagalan server	Bobot log dipisah: 4xx dicatat sebagai peringatan satu baris, 5xx tetap lengkap dengan stack trace. Ikon <code>favicon.svg</code> juga ditambahkan dan dirujuk eksplisit dari <code><head></code>
8	Badge status pada seluruh tabel selalu berwarna abu-abu, dan halaman detail transaksi yang masih dipinjam justru menampilkan keterangan "sudah dikembalikan"	Templat memanggil method entitas secara langsung. Handlebars 4.6+ memblokir akses ke anggota di prototype (<code>{{#if loan.isOngoing}}</code> bernilai <code>undefined</code>), dan argumen helper menerima FUNGSI-nya, bukan hasil pemanggilan (<code>{{statusColor this.displayStatus}}</code>)	Ditambahkan method <code>toView()</code> pada entitas <code>Loan</code> yang mengubah hasil method domain menjadi properti biasa; seluruh controller memetakan entitas melalui method ini sebelum mengirimkannya ke templat
9	Halaman error menampilkan pesan campur dua bahasa ("Error 404" dengan "Cannot GET /alamat"), memajang perintah <code>docker compose logs -f app</code> kepada pengguna akhir, dan berpotensi membocorkan pesan error basis data	Penangkap error meneruskan <code>exception.message</code> apa adanya ke tampilan, termasuk pesan bawaan framework berbahasa Inggris dan pesan error internal PostgreSQL	Ditambahkan pemetaan judul dan keterangan berbahasa Indonesia per kode status; pesan bawaan framework disaring, error 5xx tidak pernah menampilkan pesan aslinya, dan blok perintah Docker dihapus dari tampilan

2.2 Mekanisme penanganan error

Aplikasi memasang **penangkap error terpusat** (`AllExceptionsFilter`) yang mencatat setiap error ke log server lengkap dengan metode HTTP, URL, dan stack trace, lalu menampilkan halaman error yang ramah kepada pengguna. Log dapat ditelusuri dengan perintah:

```
docker compose logs -f app
```

2.3 Hasil pengujian unit otomatis

Pengujian dijalankan dengan `bun run test` dan seluruhnya berhasil:

```
PASS src/common/error.spec.ts
PASS src/common/date.spec.ts
PASS src/book/book.entity.spec.ts
PASS src/loan/loan.service.spec.ts
```

```
Test Suites: 4 passed, 4 total
Tests:       55 passed, 55 total
```

Cakupan pengujian:

Berkas Uji	Jml	Yang diuji
<code>loan.service.spec.ts</code>	26	Perhitungan denda (tepat waktu, lebih awal, terlambat, lintas bulan, tarif berbeda), pembentukan kode transaksi, perilaku entitas Loan, pengaruh perubahan status terhadap stok, dan penyiapan data untuk templat (<code>toView</code>)
<code>book.entity.spec.ts</code>	8	Aturan stok: tidak boleh negatif, tidak boleh melebihi stok total, konsistensi setelah rangkaian pinjam–kembali
<code>date.spec.ts</code>	13	Selisih hari, penambahan hari, normalisasi jam, format tanggal Indonesia, format Rupiah
<code>error.spec.ts</code>	8	Ketahanan penanganan error terhadap nilai selain <code>Error</code> (string, angka, null, objek biasa)

2.4 Hasil pengujian manual (black box)

No	Skenario	Hasil yang Diharapkan	Status
1	Buka seluruh 7 halaman	Semua merespons HTTP 200	Sesuai
2	Simpan buku dengan form kosong	Muncul 5 pesan validasi, data tidak tersimpan	Sesuai
3	Simpan buku dengan kode duplikat	Ditolak dengan pesan "Kode buku sudah digunakan"	Sesuai
4	Simpan buku dengan data valid	Tersimpan, dialihkan ke daftar buku	Sesuai
5	Pinjam buku	Stok tersedia berkurang 1, kode transaksi terbentuk otomatis	Sesuai
6	Pinjam dengan email tidak valid	Ditolak dengan pesan "Format email tidak valid"	Sesuai
7	Kembalikan buku terlambat 5 hari	Denda tercatat Rp 5.000, stok bertambah 1	Sesuai
8	Kembalikan buku yang sudah dikembalikan	Ditolak dengan pesan peringatan	Sesuai
8a	Ubah status Dipinjam → Dikembalikan	Tanggal kembali terisi, denda dihitung, stok +1	Sesuai
8b	Perpanjang jatuh tempo pada transaksi selesai	Denda dihitung ulang (Rp 3.000 → Rp 0)	Sesuai
8c	Ubah status Dikembalikan → Dipinjam	Tanggal kembali dikosongkan, denda direset, stok -1	Sesuai
8d	Ubah status ke Dipinjam saat stok habis	Ditolak, seluruh perubahan di-rollback	Sesuai
8e	Ubah jatuh tempo mendahului tanggal pinjam	Ditolak dengan pesan validasi	Sesuai
9	Hapus buku yang sedang dipinjam	Ditolak dengan pesan "sedang dipinjam"	Sesuai
10	Hapus buku yang punya riwayat transaksi selesai	Ditolak dengan pesan berbahasa Indonesia, bukan error PostgreSQL	Sesuai
11	Hapus buku tanpa riwayat transaksi	Berhasil dihapus	Sesuai
12	Cetak laporan PDF	Berkas PDF terunduh, isi tabel dan ringkasan sesuai	Sesuai
13	Kirim surel pengingat	Surel masuk ke kotak masuk Mailpit	Sesuai
14	Akses alamat yang tidak ada	Tampil "Halaman Tidak Ditemukan" berbahasa Indonesia	Sesuai
14a	Akses ID transaksi yang tidak ada	Menampilkan pesan buatan aplikasi: "Transaksi dengan ID 9999 tidak ditemukan."	Sesuai
14b	Akses ID transaksi bukan angka	Pesan bawaan framework disaring, diganti keterangan Bahasa Indonesia	Sesuai

No	Skenario	Hasil yang Diharapkan	Status
14c	Error 5xx pada sisi server	Pengguna hanya menerima keterangan umum; pesan asli PostgreSQL tetap tercatat di log	Sesuai
15	Badge status pada dashboard, tabel transaksi, dan laporan	Hijau (Dikembalikan), kuning (Dipinjam), merah (Terlambat)	Sesuai
16	Detail transaksi pada tiga kondisi berbeda	Alert kuning/merah/hijau sesuai kondisi, tombol aksi muncul hanya saat masih dipinjam	Sesuai

3. Dokumentasi Program

a. Tools Software Pemrograman yang Digunakan

Kategori	Perangkat Lunak	Versi
Framework Utama	NestJS	11.2.1
Runtime	Node.js	22 (Alpine)
Manajer Paket	Bun	1.3.14
Database Server	PostgreSQL	16 (Alpine)
Web Server	Nginx	1.27 (Alpine)
Server Surel (uji)	Mailpit	1.31.0
Kontainerisasi	Docker & Docker Compose	29.7.2 / v5.4.0
Kerangka Pengujian	Jest	29.7
Kompilator	TypeScript	5.9.3
Text Editor	Visual Studio Code	1.134
Version Control	Git	2.55

b. Bahasa Pemrograman yang Digunakan

Lapisan	Bahasa	Keterangan
Backend / Logika Inti	TypeScript 5.9	Menerapkan OOP penuh: kelas abstrak, pewarisan, enkapsulasi, interface, decorator, dan dependency injection
Basis Data	SQL (PostgreSQL)	Diakses lewat TypeORM Repository & Query Builder (prepared statement)
Tampilan	HTML5 + Handlebars (.hbs)	Server-Side Rendering
Gaya Tampilan	CSS3	286 baris gaya kustom di atas Bootstrap 5
Interaksi Peramban	JavaScript (Vanilla)	Inisialisasi grafik Chart.js
Infrastruktur	Dockerfile & YAML	Definisi container dan orkestrasi

Statistik kode: 3.176 baris TypeScript (29 berkas), 538 baris kode uji (4 berkas), 1.240 baris templat Handlebars (10 berkas), serta CSS, JavaScript, dan SVG sisi peramban (3 berkas).

c. Library, Komponen Pre-Existing, dan Framework yang Digunakan

No	Library / Komponen	Versi	Fungsi dalam Aplikasi
1	pdfmake	0.2.23	Mencetak Laporan Peminjaman menjadi berkas PDF (pdf.service.ts)
2	Nodemailer	6.10.1	Mengirim surel bukti peminjaman dan pengingat jatuh tempo (mail.service.ts)

No	Library / Komponen	Versi	Fungsi dalam Aplikasi
3	class-validator	0.14.4	Validasi input berbasis decorator pada kelas DTO
4	class-transformer	0.5.1	Konversi data mentah form HTML menjadi instance kelas DTO bertipe benar
5	TypeORM	0.3.31	Object-Relational Mapping, relasi antar tabel, dan transaksi basis data
6	@nestjs/core, common, platform-express	11.2.1	Framework aplikasi: routing, modul, dependency injection
7	@nestjs/config	4.0.4	Pembacaan variabel lingkungan (kredensial basis data & surel)
8	hbs (Handlebars)	4.2.1	Mesin templat untuk merender halaman HTML di sisi server
9	pg	8.23.0	Driver koneksi PostgreSQL
10	Bootstrap	5.3.3	Kerangka kerja CSS untuk tata letak dan komponen antarmuka
11	Bootstrap Icons	1.11.3	Kumpulan ikon antarmuka
12	Chart.js	4.5.1	Grafik batang dan grafik lingkaran pada dashboard
13	Jest + ts-jest	29.x	Kerangka kerja pengujian unit

Catatan teknis: Bootstrap, Bootstrap Icons, dan Chart.js **tidak dimuat dari CDN**, melainkan disalin ke direktori `public/vendor/` saat proses build Docker. Dengan begitu tampilan aplikasi tetap utuh walaupun lokasi uji tidak memiliki koneksi internet.

d. Dokumen Kode Program yang Dibuat

A. Lapisan Entitas & Basis Data

No	Nama Berkas	Deskripsi / Fungsi
1	<code>src/common/entities/base.entity.ts</code>	Kelas abstrak induk berisi <code>id</code> , <code>createdAt</code> , <code>updatedAt</code> . Diwarisi seluruh entitas
2	<code>src/book/book.entity.ts</code>	Entitas tabel <code>books</code> . Berisi enum <code>BookCategory</code> , relasi One-to-Many, serta method domain <code>isBorrowable()</code> , <code>decreaseStock()</code> , <code>increaseStock()</code> , <code>borrowedCount()</code>
3	<code>src/loan/loan.entity.ts</code>	Entitas tabel <code>loans</code> . Relasi Many-to-One ke Book dengan <code>onDelete: RESTRICT</code> , serta method <code>overdueDays()</code> , <code>isOverdue()</code> , <code>displayStatus()</code>

B. Lapisan Logika Bisnis (Service)

No	Nama Berkas	Deskripsi / Fungsi
4	<code>src/book/book.service.ts</code>	CRUD buku, pencarian, agregasi per kategori (<code>GROUP BY</code>), validasi stok saat perubahan data
5	<code>src/loan/loan.service.ts</code>	Inti aplikasi. Method statis <code>calculateFine()</code> , <code>buildCode()</code> , dan <code>stockEffect()</code> ; <code>borrow()</code> , <code>returnBook()</code> , dan <code>update()</code> dibungkus transaksi basis data
6	<code>src/mail/mail.service.ts</code>	Pembungkus library Nodemailer. Mengirim bukti peminjaman dan pengingat jatuh tempo
7	<code>src/report/pdf.service.ts</code>	Pembungkus library pdfmake. Menyusun struktur dokumen dan mencetaknya menjadi Buffer PDF
8	<code>src/seed/seed.service.ts</code>	Mengisi 10 buku dan 5 transaksi contoh saat aplikasi pertama dijalankan

C. Lapisan Pengendali (Controller) — 7 Halaman

No	Nama Berkas	Halaman yang Ditangani
9	<code>src/dashboard/dashboard.controller.ts</code>	Halaman 1 — Dashboard
10	<code>src/book/book.controller.ts</code>	Halaman 2 — Data Buku, Halaman 3 — Form Buku
11	<code>src/loan/loan.controller.ts</code>	Halaman 4 — Transaksi, Halaman 5 — Form Peminjaman, Halaman 6 — Detail, Halaman 8 — Form Ubah Transaksi
12	<code>src/report/report.controller.ts</code>	Halaman 7 — Laporan & Cetak PDF

D. Lapisan DTO (Data Transfer Object)

No	Nama Berkas	Deskripsi / Fungsi
13	<code>src/book/dto/create-book.dto.ts</code>	Aturan validasi penambahan buku (decorator <code>@IsNotEmpty</code> , <code>@IsInt</code> , <code>@Min</code> , <code>@Max</code> , <code>@IsEnum</code>)
14	<code>src/book/dto/update-book.dto.ts</code>	Mewarisi <code>CreateBookDto</code> dengan <code>extends</code> (prinsip DRY)
15	<code>src/loan/dto/create-loan.dto.ts</code>	Aturan validasi transaksi peminjaman termasuk <code>@IsEmail</code>
16	<code>src/loan/dto/update-loan.dto.ts</code>	Aturan validasi pengubahan transaksi: identitas, jatuh tempo (<code>@IsISO8601</code>), dan status (<code>@IsEnum</code>)

E. Modul & Titik Masuk

No	Nama Berkas	Deskripsi / Fungsi
17	<code>src/app.module.ts</code>	Modul akar. Konfigurasi koneksi PostgreSQL dari variabel lingkungan
18	<code>src/main.ts</code>	Titik masuk. Menyiapkan Handlebars, aset statis, dan penangkap error global
19–24	<code>book.module.ts</code> , <code>loan.module.ts</code> , <code>dashboard.module.ts</code> , <code>report.module.ts</code> , <code>mail.module.ts</code> , <code>seed.module.ts</code>	Pengelompokan komponen per fitur

F. Komponen Bersama (Common)

No	Nama Berkas	Deskripsi / Fungsi
25	<code>src/common/date.ts</code>	Fungsi murni: <code>startOfDay</code> , <code>endOfDay</code> , <code>addDays</code> , <code>diffInDays</code> , <code>formatDate</code> , <code>formatIsoDate</code> , <code>formatCurrency</code> , dan <code>DateTransformer</code>
26	<code>src/common/validation.ts</code>	Pembungkus class-validator + class-transformer untuk validasi form HTML
27	<code>src/common/error.ts</code>	Fungsi <code>errorMessage()</code> dan <code>errorStack()</code> — penanganan aman atas nilai bertipe <code>unknown</code> pada blok <code>catch</code>
28	<code>src/common/hbs-helpers.ts</code>	10 helper Handlebars: <code>eq</code> , <code>ne</code> , <code>gt</code> , <code>inc</code> , <code>multiply</code> , <code>formatDate</code> , <code>isoDate</code> , <code>currency</code> , <code>statusColor</code> , <code>categoryColor</code> , <code>stockColor</code>
29	<code>src/common/filters/all-exceptions.filter.ts</code>	Penangkap error terpusat — bukti unit Melakukan Debugging

G. Lapisan Tampilan (Views) & Aset

No	Nama Berkas	Deskripsi / Fungsi
30	<code>views/layouts/main.hbs</code>	Kerangka utama: sidebar, topbar, area notifikasi, footer
31	<code>views/dashboard.hbs</code>	Halaman 1 — kartu statistik, 2 grafik Chart.js, transaksi terbaru
32	<code>views/book/index.hbs</code>	Halaman 2 — tabel koleksi buku + pencarian

No	Nama Berkas	Deskripsi / Fungsi
33	<code>views/book/form.hbs</code>	Halaman 3 — form tambah dan ubah (satu templat dua fungsi)
34	<code>views/loan/index.hbs</code>	Halaman 4 — tabel transaksi + filter status
35	<code>views/loan/form.hbs</code>	Halaman 5 — form peminjaman buku
36	<code>views/loan/detail.hbs</code>	Halaman 6 — detail transaksi + aksi pengembalian
37	<code>views/loan/form-edit.hbs</code>	Halaman 8 — form ubah transaksi (identitas, jatuh tempo, status)
38	<code>views/report/index.hbs</code>	Halaman 7 — filter periode, cetak PDF, kirim pengingat
39	<code>views/error.hbs</code>	Halaman error
40	<code>public/css/app.css</code>	Gaya kustom aplikasi (tata letak sidebar, kartu, tabel)
41	<code>public/js/dashboard.js</code>	Skrip Chart.js halaman dashboard, dipisahkan dari templat agar tetap berupa JavaScript murni
42	<code>public/img/favicon.svg</code>	Ikon tab peramban, dirujuk eksplisit dari <code><head></code>

H. Pengujian Unit

No	Nama Berkas	Jml Uji
43	<code>src/loan/loan.service.spec.ts</code>	26
44	<code>src/common/date.spec.ts</code>	13
45	<code>src/book/book.entity.spec.ts</code>	8
46	<code>src/common/error.spec.ts</code>	8

I. Konfigurasi & Infrastruktur

No	Nama Berkas	Deskripsi / Fungsi
47	<code>docker-compose.yml</code>	Orkestrasi 4 container: nginx, app, db, mailpit
48	<code>Dockerfile</code>	Build 4 tahap: dependensi (Bun) → dependensi produksi → kompilasi → image akhir
49	<code>nginx/default.conf</code>	Konfigurasi web server sebagai reverse proxy ke port 3000
50	<code>jest.config.js</code>	Konfigurasi kerangka pengujian
51	<code>tsconfig.json</code>	Konfigurasi kompilator TypeScript (decorator, metadata, dan <code>useUnknownInCatchVariables</code> aktif)
52	<code>.prettierrc</code> , <code>.prettierrignore</code>	Penyeragaman gaya penulisan kode; direktori <code>views/</code> dikecualikan agar sintaks templat tidak rusak

4. Struktur Basis Data

Tabel `books`

Kolom	Tipe	Keterangan
<code>id</code>	integer	Primary key, auto increment
<code>code</code>	varchar(20)	UNIQUE — contoh: BK-001
<code>title</code>	varchar(150)	NOT NULL — judul buku
<code>author</code>	varchar(100)	NOT NULL — pengarang
<code>publisher</code>	varchar(100)	NULL — penerbit
<code>published_year</code>	integer	NOT NULL — tahun terbit

Kolom	Tipe	Keterangan
<code>category</code>	enum	Umum, Teknologi, Sains, Sejarah, Fiksi
<code>stock</code>	integer	Total eksemplar
<code>available</code>	integer	Eksemplar yang siap dipinjam
<code>created_at, updated_at</code>	timestamp	Kolom audit dari <code>BaseEntity</code>

Tabel `loans`

Kolom	Tipe	Keterangan
<code>id</code>	integer	Primary key, auto increment
<code>code</code>	varchar(30)	UNIQUE — contoh: PJM-20260825-001
<code>borrower_name</code>	varchar(100)	NOT NULL — nama peminjam
<code>borrower_email</code>	varchar(120)	NOT NULL — surel peminjam
<code>book_id</code>	integer	FOREIGN KEY → <code>books(id)</code> ON DELETE RESTRICT
<code>borrowed_at</code>	date	NOT NULL — tanggal pinjam
<code>due_at</code>	date	NOT NULL — tanggal jatuh tempo
<code>returned_at</code>	date	NULL selama buku belum dikembalikan
<code>status</code>	enum	Dipinjam, Dikembalikan
<code>fine</code>	integer	Denda keterlambatan dalam rupiah
<code>created_at, updated_at</code>	timestamp	Kolom audit dari <code>BaseEntity</code>

Relasi: satu baris `books` memiliki banyak baris `loans` (One-to-Many); satu baris `loans` merujuk tepat satu baris `books` (Many-to-One). Batasan **ON DELETE RESTRICT** mencegah penghapusan buku yang masih memiliki riwayat transaksi, sehingga integritas referensial terjaga di tingkat basis data.

Catatan penamaan. Nama tabel dan kolom memakai Bahasa Inggris mengikuti konvensi penulisan kode, sedangkan NILAI yang tersimpan di kolom `category` dan `status` tetap Bahasa Indonesia karena nilai itulah yang langsung ditampilkan kepada pengguna.

5. Penerapan Pemrograman Berorientasi Objek

Konsep OOP	Penerapan Nyata dalam Kode
Kelas & Objek	Seluruh berkas <code>.entity.ts</code> , <code>.service.ts</code> , <code>.controller.ts</code> , <code>.dto.ts</code>
Abstraksi	<code>BaseEntity</code> dideklarasikan <code>abstract</code> dan tidak pernah diinstansiasi langsung
Pewarisan	<code>Book extends BaseEntity</code> , <code>Loan extends BaseEntity</code> , <code>UpdateBookDto extends CreateBookDto</code>
Enkapsulasi	Stok hanya boleh diubah lewat <code>decreaseStock()/increaseStock()</code> ; atribut <code>private readonly logger</code> dan method <code>private</code> pada service
Polimorfisme	<code>displayStatus()</code> menghasilkan label berbeda ("Dipinjam" vs "Terlambat") untuk data status yang sama, bergantung kondisi jatuh tempo
Pemisahan Lapisan	<code>toView()</code> mengubah entitas menjadi objek biasa berisi hasil perhitungan, sehingga lapisan tampilan tidak perlu memanggil method domain
Interface	<code>AllExceptionsFilter implements ExceptionFilter</code> , <code>MailService implements OnModuleInit</code> , <code>SeedService implements OnApplicationBootstrap</code>

Konsep OOP	Penerapan Nyata dalam Kode
Method Statis	<code>LoanService.calculateFine()</code> , <code>.buildCode()</code> , dan <code>.stockEffect()</code> — milik kelas, bukan instance
Dependency Injection	Seluruh Repository dan Service disuntikkan lewat constructor, bukan dibuat sendiri di dalam kelas
Decorator	<code>@Entity</code> , <code>@Column</code> , <code>@ManyToOne</code> , <code>@Injectable</code> , <code>@Controller</code> , <code>@Get</code> , <code>@Post</code> , <code>@IsEmpty</code>

6. Penerapan Akses Basis Data

Teknik	Berkas	Contoh
Repository Pattern	<code>book.service.ts</code>	<code>this.bookRepository.find({ order: { code: 'ASC' } })</code>
Prepared Statement	seluruh service	Parameter selalu diikat (<code>:status</code> , <code>:year</code>) — aman dari SQL Injection
Query Builder + JOIN	<code>loan.service.ts</code>	<code>.leftJoinAndSelect('l.book', 'b')</code>
Agregasi & GROUP BY	<code>book.service.ts</code>	<code>SUM(b.stock)</code> dikelompokkan per kategori
Fungsi SQL	<code>loan.service.ts</code>	<code>EXTRACT(MONTH FROM p.tanggal_pinjam)</code> untuk grafik bulanan
Transaksi ACID	<code>loan.service.ts</code>	<code>this.dataSource.transaction(...)</code> membungkus pengurangan stok + penyimpanan transaksi; bila salah satu gagal, seluruhnya dibatalkan
Value Transformer	<code>common/date.ts</code>	Konversi kolom <code>date</code> PostgreSQL ↔ objek <code>Date</code> JavaScript

7. Cara Menjalankan Aplikasi

```
docker compose up --build
```

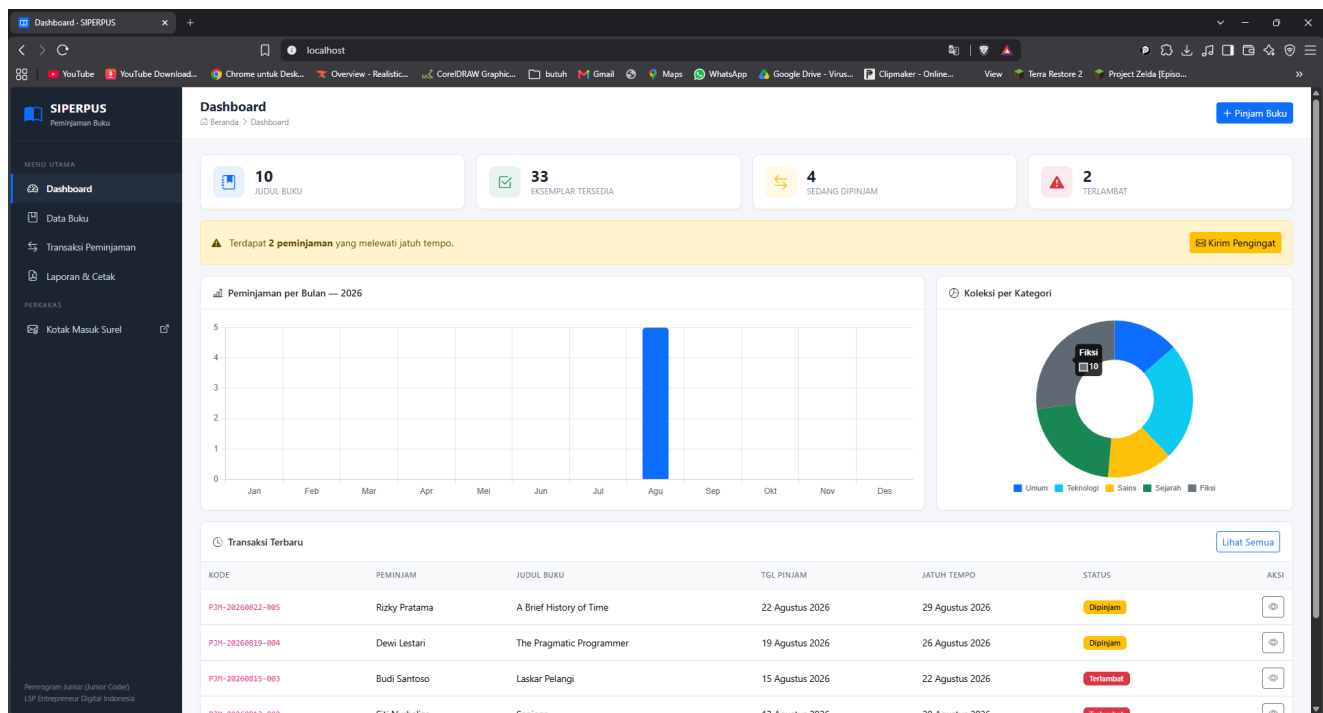
Alamat	Keterangan
<code>http://localhost</code>	Aplikasi (melalui web server Nginx)
<code>http://localhost:8025</code>	Kotak masuk surel (Mailpit)
<code>localhost:5432</code>	PostgreSQL — user/password/database: <code>siperpus</code>

Menjalankan pengujian unit:

```
bun install && bun run test
```

8. Tampilan Aplikasi

Berikut tangkapan layar seluruh halaman aplikasi dan bukti berjalannya setiap fitur yang telah dibangun.



Gambar 1. Halaman 1 — Dashboard. Kartu statistik, grafik batang peminjaman per bulan dan grafik lingkaran komposisi koleksi (Chart.js), serta daftar transaksi terbaru.

Data Buku

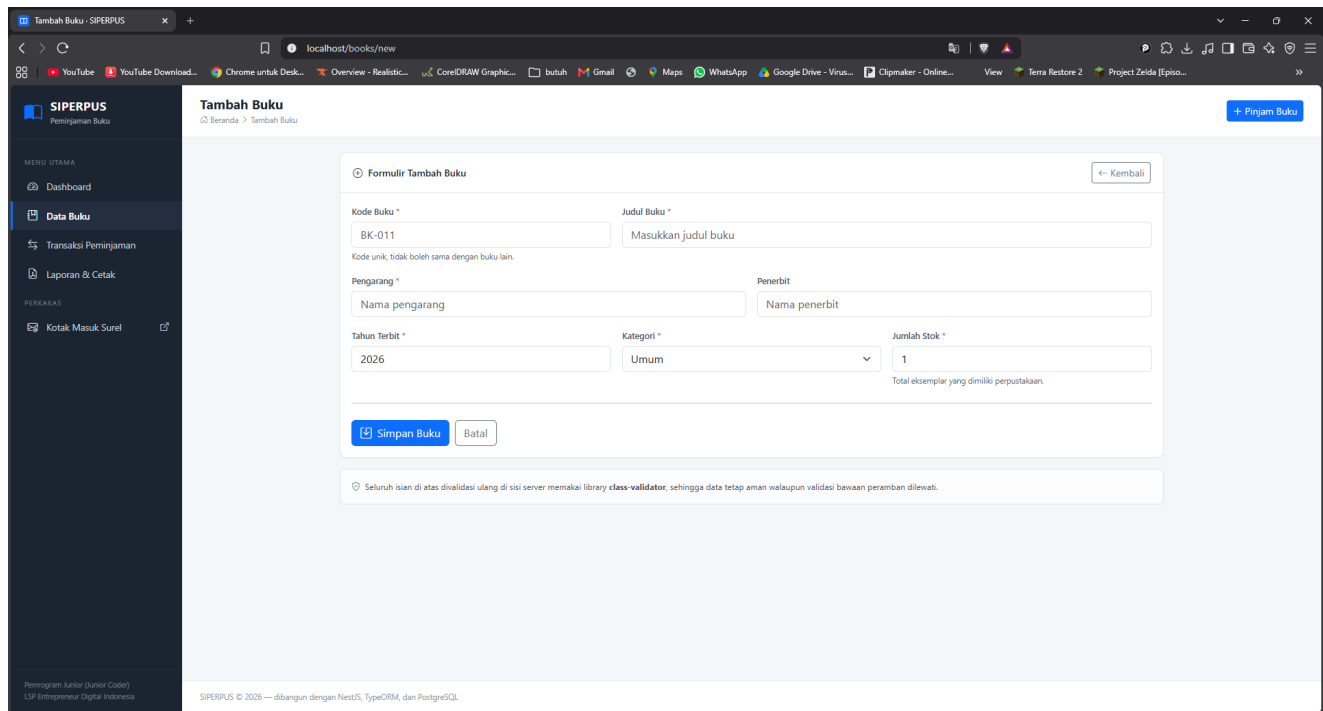
Koleksi Buku: 10 judul

Cari judul, pengarang, atau kode... [+ Tambah Buku](#)

NO	KODE	JUDUL	PENGARANG	PENERBIT	TAHUN	KATEGORI	STOK	TERSEDIA	AKSI
1	BK-001	Clean Code	Robert C. Martin	Prentice Hall	2008	Teknologi	4	4	Edit Hapus
2	BK-002	The Pragmatic Programmer	Andrew Hunt	Addison-Wesley	1999	Teknologi	3	2	Edit Hapus
3	BK-003	Design Patterns	Erich Gamma	Addison-Wesley	1994	Teknologi	2	2	Edit Hapus
4	BK-004	Sapiens	Yuval Noah Harari	Harper	2011	Sejarah	5	4	Edit Hapus
5	BK-005	A Brief History of Time	Stephen Hawking	Bantam Books	1988	Sains	3	2	Edit Hapus
6	BK-006	Cosmos	Carl Sagan	Random House	1980	Sains	2	2	Edit Hapus
7	BK-007	Laskar Pelangi	Andrea Hirata	Bentang Pustaka	2005	Fiksi	6	5	Edit Hapus
8	BK-008	Bumi Manusia	Pramoedya Ananta Toer	Hasta Mitra	1980	Fiksi	4	4	Edit Hapus
9	BK-009	Sejarah Indonesia Modern	M.C. Ricklefs	Serambi	2008	Sejarah	3	3	Edit Hapus
10	BK-010	Filosofi Teras	Henry Manampiring	Kompas	2018	Umum	5	5	Edit Hapus

SIPERUS © 2026 — dibangun dengan NextJS, TypeORM, dan PostgreSQL.

Gambar 2. Halaman 2 — Data Buku (/books). Tabel koleksi dengan pencarian, lencana kategori, indikator stok, serta tombol ubah dan hapus.



Tambah Buku

Formulir Tambah Buku

Kode Buku * BK-011 Judul Buku * Masukkan judul buku

Kode unik, tidak boleh sama dengan buku lain.

Pengarang * Nama pengarang Penerbit Nama penerbit

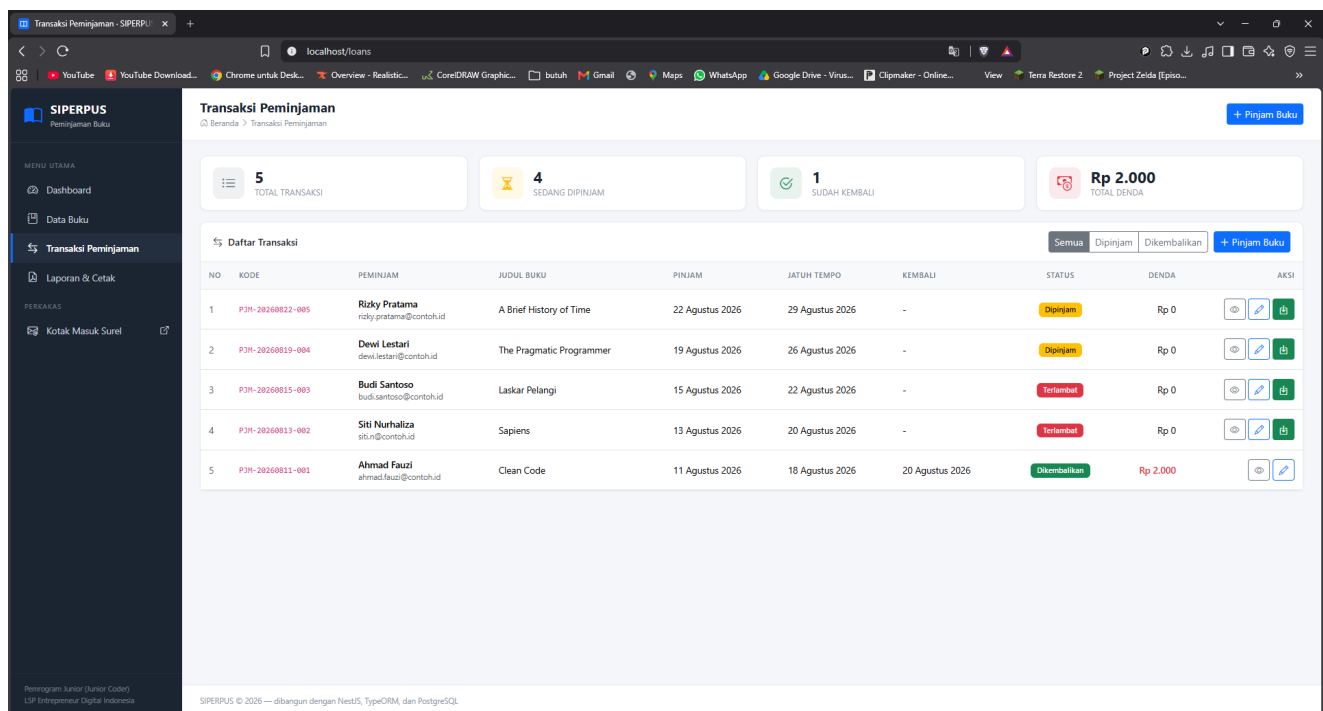
Tahun Terbit * 2026 Kategori * Umum Jumlah Stok * 1

Total eksemplar yang dimiliki perpustakaan

Simpan Buku Batal

Seluruh isian di atas divalidasi ulang di sisi server memakai library class-validator, sehingga data tetap aman walaupun validasi bawaan peramban dilewati.

Gambar 3. Halaman 3 — Form Buku (/books/new). Satu templat dipakai untuk mode tambah maupun ubah.



Transaksi Peminjaman

5 TOTAL TRANSAKSI 4 SEDANG DIPINJAM 1 SUDAH KEMBALI Rp 2.000 TOTAL DENDA

Daftar Transaksi

NO	KODE	PEMINJAM	JUDUL BUKU	PINJAM	JATUH TEMPO	KEMBALI	STATUS	DENDA	AKSI
1	PJH-20260822-005	Rizky Pratama rizky.pratama@contoh.id	A Brief History of Time	22 Agustus 2026	29 Agustus 2026	-	Dipinjam	Rp 0	[Edit] [Hapus] [Detail]
2	PJH-20260819-004	Dewi Lestari dewi.lestari@contoh.id	The Pragmatic Programmer	19 Agustus 2026	26 Agustus 2026	-	Dipinjam	Rp 0	[Edit] [Hapus] [Detail]
3	PJH-20260815-003	Budi Santoso budi.santoso@contoh.id	Laskar Pelangi	15 Agustus 2026	22 Agustus 2026	-	Terlambat	Rp 0	[Edit] [Hapus] [Detail]
4	PJH-20260813-002	Siti Nurhaliza siti.n@contoh.id	Sapiens	13 Agustus 2026	20 Agustus 2026	-	Terlambat	Rp 0	[Edit] [Hapus] [Detail]
5	PJH-20260811-001	Ahmad Fauzi ahmad.fauzi@contoh.id	Clean Code	11 Agustus 2026	18 Agustus 2026	20 Agustus 2026	Dikembalikan	Rp 2.000	[Edit] [Hapus] [Detail]

Gambar 4. Halaman 4 — Transaksi Peminjaman (/loans). Ringkasan angka, filter status, dan lencana berwarna sesuai kondisi transaksi.

Pinjam Buku
Beranda > Pinjam Buku

Formulir Peminjaman Buku [← Kembali](#)

Nama Peminjam *
Nama lengkap peminjam

Email Peminjam *
nama@contoh.id
Buku peminjaman dikirim otomatis ke alamat ini.

Buku yang Dipinjam *
-- Pilih buku --

Lama Peminjaman *
7 hari

Hanya buku dengan stok tersedia yang ditampilkan.

ⓘ Keterlambatan pengembalian dikenakan denda sebesar Rp 1.000 per hari, dihitung otomatis oleh sistem saat buku dikembalikan.

[Proses Peminjaman](#) [Batal](#)

SIPERPUS © 2026 — dibangun dengan NextJS, TypeORM, dan PostgreSQL

Gambar 5. Halaman 5 — Form Peminjaman (/loans/new). Hanya buku dengan stok tersedia yang muncul pada daftar pilihan.

Detail Peminjaman
Beranda > Detail Peminjaman

Transaksi P3H-20260815-003 [Terlambat](#)

NAMA PEMINJAM
Budi Santoso

EMAIL PEMINJAM
budi.santoso@contoh.id

TANGGAL PINJAM
15 Agustus 2026

JATUH TEMPO
22 Agustus 2026

TANGGAL KEMBALI
-

KETERLAMBATAN
3 hari

DENDA
Rp 0
Perkiraan denda bila dikembalikan hari ini: Rp 3.000

[Proses Pengembalian](#) [Kirim Surel Pengingat](#) [Buku Laskar Pelangi terlambat dikembalikan 3 hari.](#) [Ubah Transaksi](#)

[← Kembali ke Daftar](#)

Buku yang Dipinjam

Laskar Pelangi
BK-007

Pengarang
Andrea Hirata

Penerbit
Bentang Pustaka

Tahun
2005

Kategori
Fiksi

Stok Tersedia
5 / 6

SIPERPUS © 2026 — dibangun dengan NextJS, TypeORM, dan PostgreSQL

Gambar 6. Halaman 6 — Detail Peminjaman (/loans/3). Transaksi yang melewati jatuh tempo ditandai merah beserta perkiraan dendanya.

Laporan Peminjaman

Periode Laporan

Tanggal Awal: 26/07/2026 | Tanggal Akhir: 25/08/2026 | [Tampilkan](#) | [Atur Ulang](#)

5 TRANSAKSI | 4 SEDANG DIPINJAM | 1 SUDAH KEMBALI | **Rp 2.000** TOTAL DENDA

[Cetak Laporan PDF](#) | [Unduh PDF](#) | [Kirim Suren Peningkat](#)

Pratinjau Laporan (5 baris data)

NO	KODE	PEMINJAM	JUDUL BUKU	PINJAM	JATUH TEMPO	KEMBALI	STATUS	DENDA
1	PJM-20260811-001	Ahmad Fauzi	Clean Code	11 Agustus 2026	18 Agustus 2026	20 Agustus 2026	Dikembalikan	Rp 2.000
2	PJM-20260813-002	Siti Nurhaliza	Sapiens	13 Agustus 2026	20 Agustus 2026	-	Tertambat	Rp 0
3	PJM-20260815-003	Budi Santoso	Laskar Pelangi	15 Agustus 2026	22 Agustus 2026	-	Tertambat	Rp 0
4	PJM-20260819-004	Dewi Lestari	The Pragmatic Programmer	19 Agustus 2026	26 Agustus 2026	-	Dipinjam	Rp 0
5	PJM-20260822-005	Rizky Pratama	A Brief History of Time	22 Agustus 2026	29 Agustus 2026	-	Dipinjam	Rp 0

Gambar 7. Halaman 7 — Laporan & Cetak (/reports). Filter periode, tombol unduh PDF, dan tombol kirim surel pengingat.

Ubah Transaksi Peminjaman

☒ Ubah Transaksi PJM-20260815-003 | [← Kembali](#)

BUKU YANG DIPINJAM: Laskar Pelangi | BIL: 007 | TANGGAL PINJAM: 15 Agustus 2026 | STOK BUKU: 5 / 6 | DENDA SAAT INI: Rp 0

Nama Peminjam: Budi Santoso | Email Peminjam: budi.santoso@contoh.id

Tanggal Jatuh Tempo: 22/08/2026 | Status Peminjaman: **Dipinjam**

Majukan tanggal ini untuk memperpanjang masa pinjam. Denda dihitung ulang otomatis setelah disimpan. | Mengubah status ikut menyesuaikan stok buku.

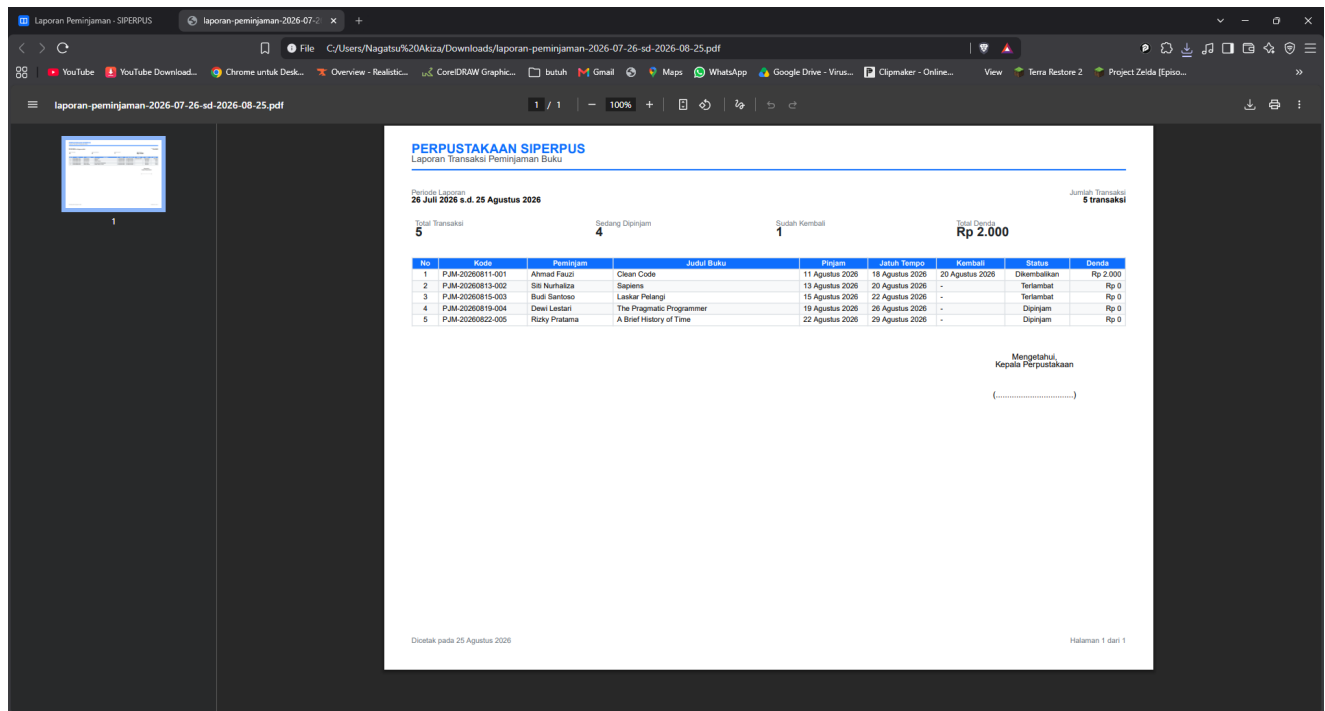
⚠️ Pengaruh perubahan status terhadap stok

- Dipinjam → Dikembalikan** — buku kembali ke rak, stok tersedia **bertambah 1**, tanggal kembali diisi hari ini, dan denda dihitung ulang.
- Dikembalikan → Dipinjam** — buku keluar lagi dari rak, stok tersedia **berkurang 1**, tanggal kembali dikosongkan, dan denda diset menjadi Rp 0. Ditolak bila stok sedang habis.

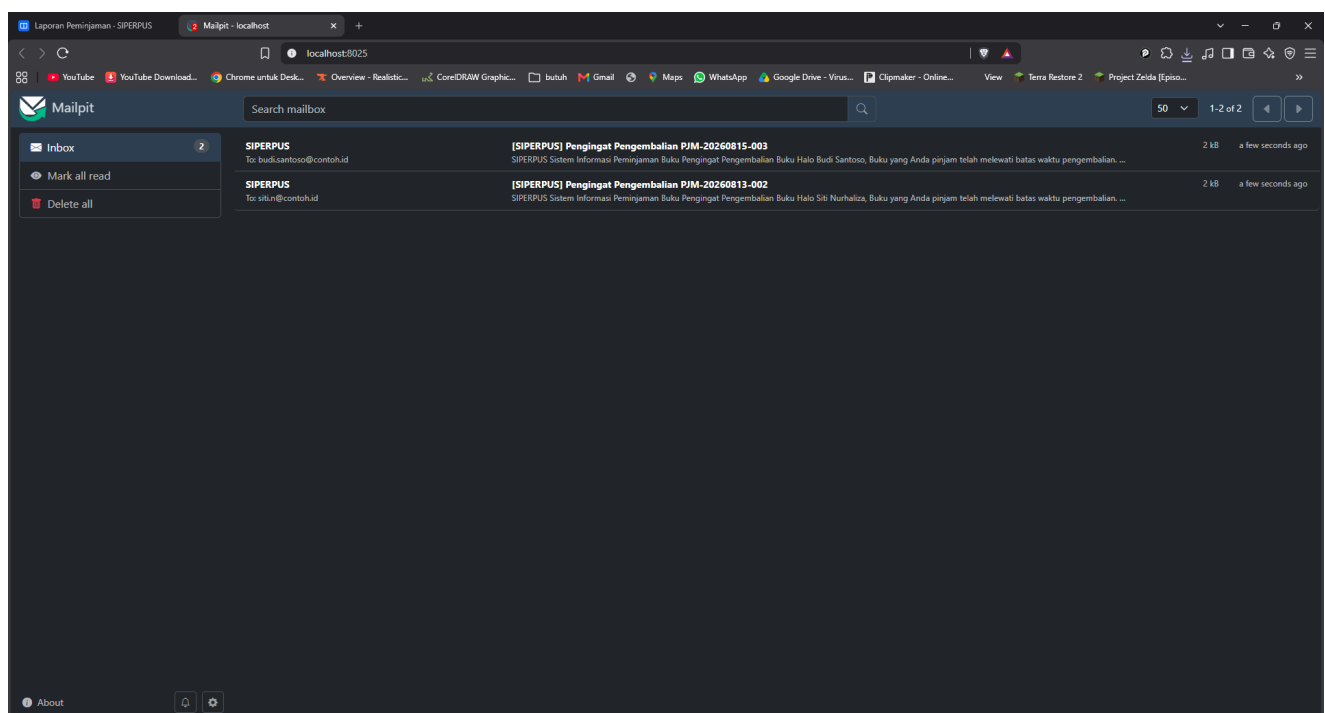
[Simpan Perubahan](#) | [Batal](#)

⚠️ Seluruh perubahan dijalankan dalam satu **transaksi basis data**. Bila salah satu langkah gagal — misalnya stok tidak mencukupi — seluruh perubahan dibatalkan sehingga data tidak pernah setengah jadi.

Gambar 8. Halaman 8 — Form Ubah Transaksi (/loans/3/edit). Pengubahan identitas, jatuh tempo, dan status peminjaman.



Gambar 9. Berkas PDF hasil cetak laporan, dibuat memakai library pdfmake.

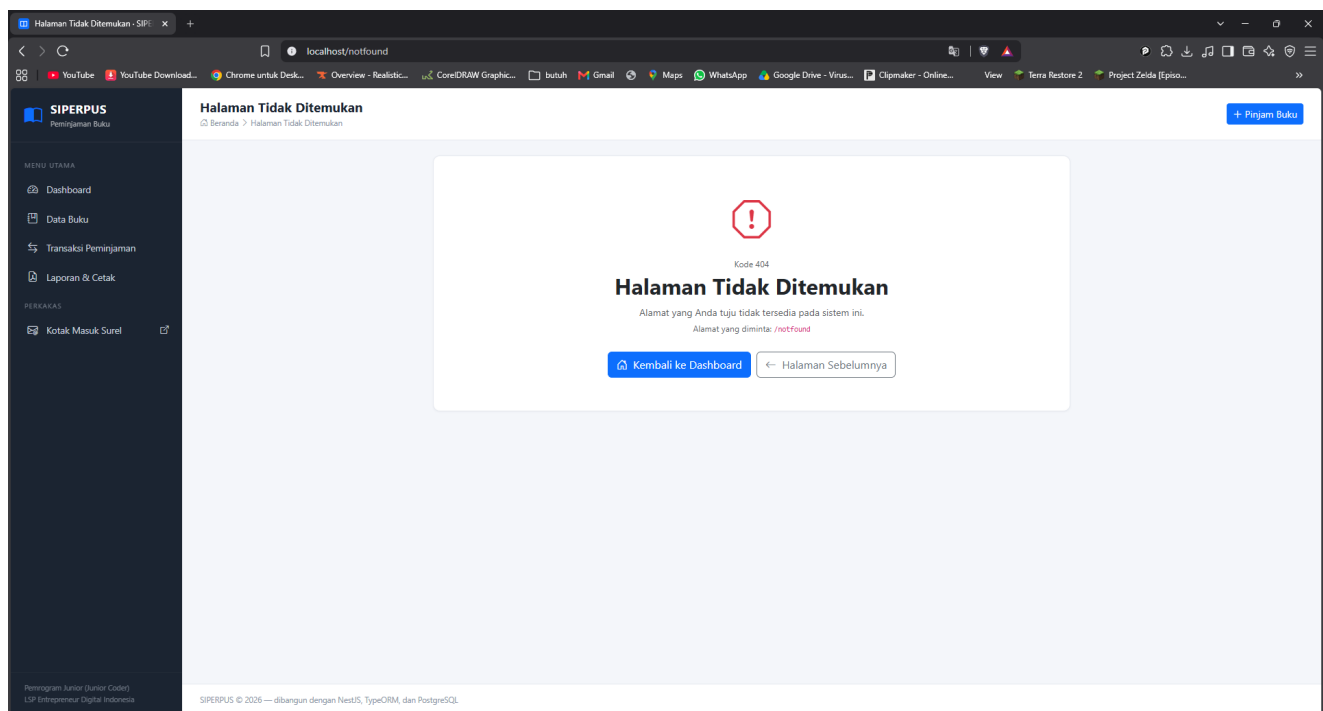


Gambar 10. Kotak masuk Mailpit berisi surel yang dikirim aplikasi lewat Nodemailer.

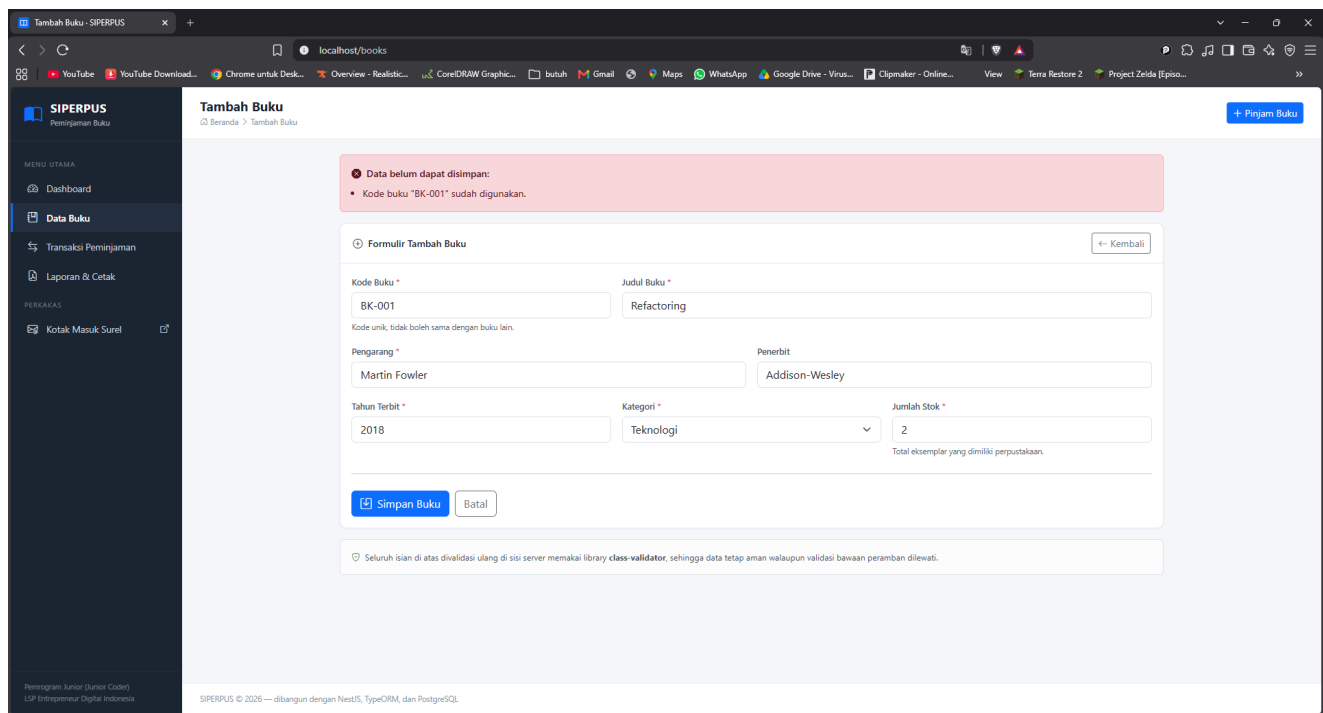
```
> cd d:\project\lspedi\
> bun run test
$ jest
PASS src/common/date.spec.ts (5.52 s)
PASS src/common/error.spec.ts (5.536 s)
PASS src/book/book.entity.spec.ts (6.451 s)
PASS src/loan/loan.service.spec.ts (7.368 s)

Test Suites: 4 passed, 4 total
Tests:       55 passed, 55 total
Snapshots:   0 total
Time:        8.532 s, estimated 18 s
Ran all test suites.
```

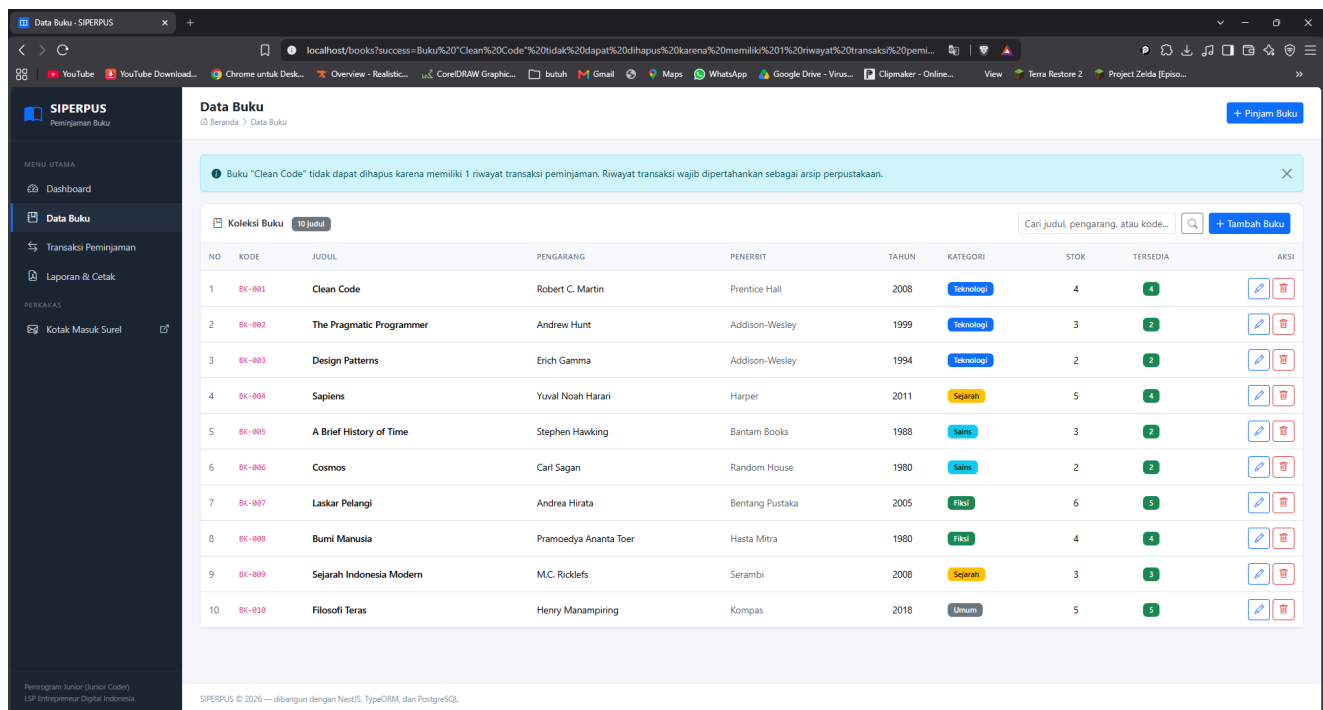
Gambar 11. Hasil perintah bun run test — 55 pengujian lolos pada 4 berkas spec.



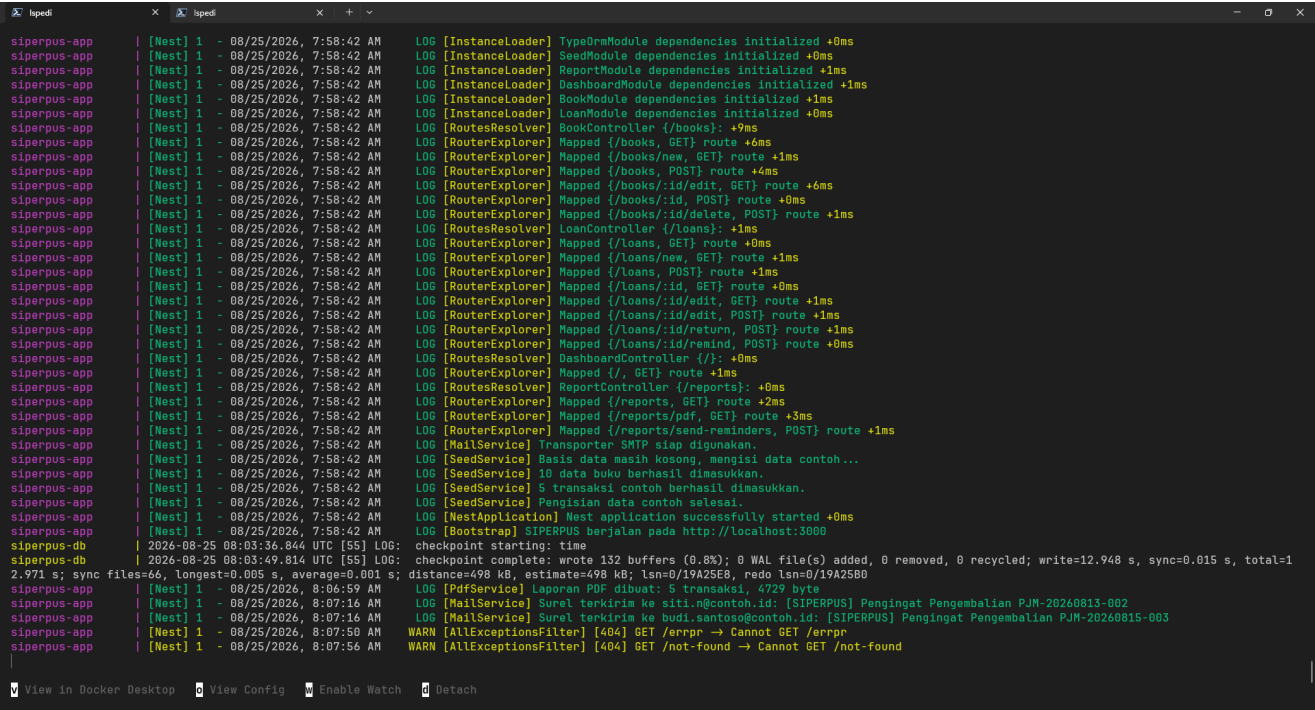
Gambar 12. Halaman error 404 berbahasa Indonesia, tanpa membocorkan pesan teknis.



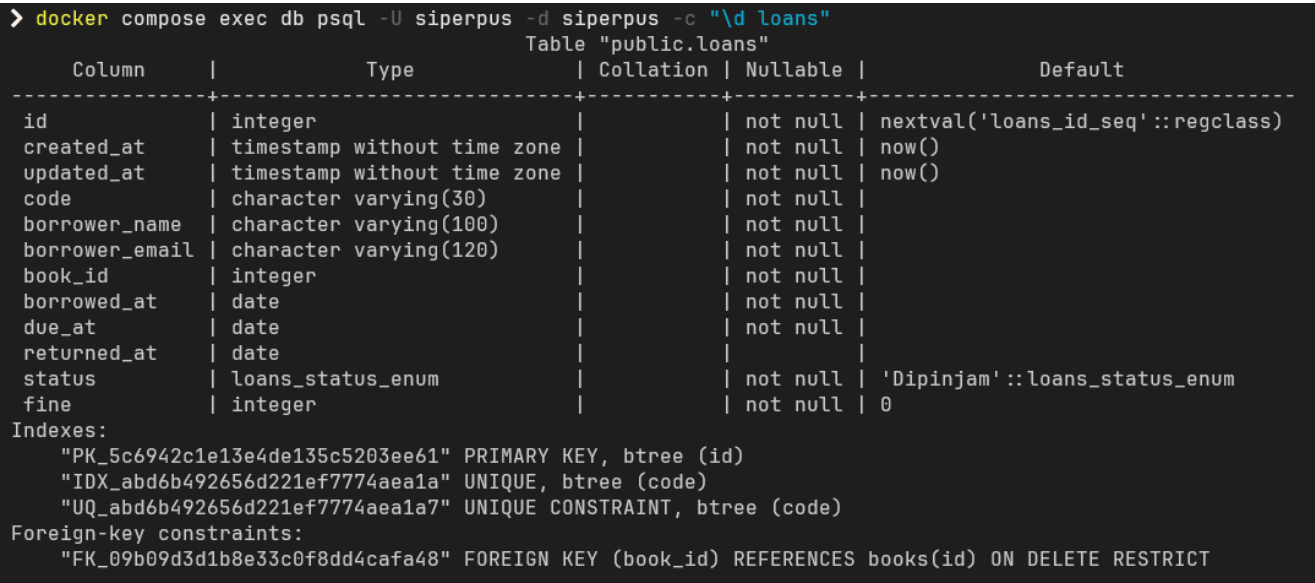
Gambar 13. Validasi sisi server menolak penyimpanan buku berkode ganda. Isian sudah lengkap sehingga lolos pemeriksaan peramban, namun ditolak aplikasi.



Gambar 14. Penolakan penghapusan buku yang masih memiliki riwayat transaksi.



Gambar 15. Empat container berjalan: nginx, app, db, dan mailpit.



Gambar 16. Struktur tabel loans beserta indeks dan foreign key ke tabel books.

9. Potongan Source Code (CRUD dan Library)

Bagian ini menampilkan potongan kode inti sebagai bukti penerapan unit kompetensi. Kode lengkapnya tersedia pada repository di bagian 10.

Catatan penamaan. Seluruh pengenalan di dalam kode (nama kelas, method, variabel, berkas, tabel, dan kolom) memakai Bahasa Inggris mengikuti konvensi penulisan kode. Sementara itu komentar, pesan validasi, dan seluruh teks yang tampil di layar tetap Bahasa Indonesia.

9.1 CREATE — Menyimpan Data Buku

```
// src/book/book.service.ts
async create(dto: CreateBookDto): Promise<Book> {
  const duplicate = await this.bookRepository.findOne({
    where: { code: dto.code },
  });
  if (duplicate) {
    throw new BadRequestException(`Kode buku "${dto.code}" sudah digunakan.`);
  }

  const book = this.bookRepository.create({ ...dto, available: dto.stock });
  const saved = await this.bookRepository.save(book);
  this.logger.log(`Buku baru tersimpan: ${saved.code} - ${saved.title}`);
  return saved;
}
```

Keterangan:

- `findOne()` — memeriksa keunikan kode buku sebelum penyimpanan.
- `create()` — membentuk instance entitas `Book` dari DTO.
- `save()` — menjalankan perintah `INSERT` sebagai prepared statement.
- Saat pertama dibuat, `available` disamakan dengan `stock` karena belum ada eksemplar yang dipinjam.

9.2 READ — Membaca Data dengan Pencarian dan Agregasi

```
// src/book/book.service.ts
async findAll(search?: string): Promise<Book[]> {
  if (search && search.trim() !== '') {
    const keyword = `%${search.trim()}%`;
    return this.bookRepository.find({
      where: [
        { title: ILike(keyword) },
        { author: ILike(keyword) },
        { code: ILike(keyword) },
      ],
      order: { code: 'ASC' },
    });
  }
  return this.bookRepository.find({ order: { code: 'ASC' } });
}

async countByCategory(): Promise<{ category: string; total: number }[]> {
  const rows = await this.bookRepository
    .createQueryBuilder('b')
    .select('b.category', 'category')
    .addSelect('SUM(b.stock)', 'total')
    .groupBy('b.category')
    .orderBy('b.category', 'ASC')
    .getRawMany();

  return rows.map((row) => ({
    category: row.category,
    total: Number(row.total),
  }));
}
```

Keterangan:

- **ILike** — pencarian tanpa membedakan huruf besar/kecil pada tiga kolom.
- Array pada **where** bermakna **OR**, sedangkan objek bermakna **AND**.
- Query Builder dipakai untuk agregasi **SUM** dengan **GROUP BY**, yang hasilnya menjadi sumber data grafik lingkaran di dashboard.

9.3 READ — Relasi Antar Tabel (JOIN)

```
// src/loan/loan.service.ts
async findAll(status?: string): Promise<Loan[]> {
  const query = this.loanRepository
    .createQueryBuilder('l')
    .leftJoinAndSelect('l.book', 'b')
    .orderBy('l.id', 'DESC');

  if (status === 'borrowed') {
    query.where('l.status = :status', { status: LoanStatus.BORROWED });
  } else if (status === 'returned') {
    query.where('l.status = :status', { status: LoanStatus.RETURNED });
  }

  return query.getMany();
}
```

Keterangan:

- **leftJoinAndSelect** — menggabungkan tabel **loans** dan **books** dalam satu kueri, sehingga judul buku tampil tanpa perlu kueri tambahan.
- **:status** adalah **parameter terikat** (prepared statement), bukan penyambungan string, sehingga aman dari serangan SQL Injection.

9.4 CREATE — Transaksi Peminjaman dengan Transaksi Basis Data

```
// src/loan/loan.service.ts
async borrow(dto: CreateLoanDto): Promise<Loan> {
  return this.dataSource.transaction(async (manager) => {
    const book = await manager.findOne(Book, { where: { id: dto.bookId } });

    if (!book) {
      throw new NotFoundException('Buku yang dipilih tidak ditemukan.');
```

```
    }
    if (!book.isBorrowable()) {
      throw new BadRequestException(
        `Stok buku "${book.title}" sedang habis dipinjam.`
      );
    }

    book.decreaseStock(); // method domain milik entitas Book
    await manager.save(Book, book);

    const borrowedAt = new Date();
    const loan = manager.create(Loan, {
      code: LoanService.buildCode(borrowedAt, todayCount + 1),
      borrowerName: dto.borrowerName,
      borrowerEmail: dto.borrowerEmail,
      bookId: book.id,
      borrowedAt,
      dueAt: addDays(borrowedAt, dto.loanDays),
      status: LoanStatus.BORROWED,
      fine: 0,
    });

    return manager.save(Loan, loan);
  });
}
```

Keterangan:

- `dataSource.transaction()` — membungkus dua operasi tulis (perubahan stok buku dan penyimpanan transaksi) menjadi satu kesatuan **ACID**.
- Bila salah satu langkah gagal, seluruh perubahan dibatalkan (**rollback**), sehingga stok buku tidak pernah berkurang tanpa transaksi pasangannya.
- `book.decreaseStock()` — pemanggilan **method domain** milik entitas, bukan pengubahan atribut secara langsung (penerapan enkapsulasi).

9.5 UPDATE — Mengubah Status Peminjaman

```

// src/loan/loan.service.ts

/** Method STATIS – murni, tidak menyentuh basis data, mudah diuji. */
static stockEffect(
  oldStatus: LoanStatus,
  newStatus: LoanStatus,
): 'increase' | 'decrease' | 'none' {
  if (oldStatus === newStatus) {
    return 'none';
  }
  return newStatus === LoanStatus.RETURNED ? 'increase' : 'decrease';
}

async update(id: number, dto: UpdateLoanDto): Promise<Loan> {
  return this.dataSource.transaction(async (manager) => {
    const loan = await manager.findOne(Loan, {
      where: { id },
      relations: { book: true },
    });

    const newDueAt = new Date(`${dto.dueAt}T00:00:00`);
    if (diffInDays(loan.borrowedAt, newDueAt) < 0) {
      throw new BadRequestException(
        'Tanggal jatuh tempo tidak boleh mendahului tanggal peminjaman.',
      );
    }

    const effect = LoanService.stockEffect(loan.status, dto.status);

    if (effect !== 'none') {
      const book = await manager.findOne(Book, {
        where: { id: loan.bookId },
      });

      if (effect === 'increase') {
        book.increaseStock();
        loan.returnedAt = new Date();
      } else {
        if (!book.isBorrowable()) {
          throw new BadRequestException(
            `Status tidak dapat dikembalikan menjadi "Dipinjam" karena stok ` +
            `buku "${book.title}" sedang habis.`
          );
        }
        book.decreaseStock();
        loan.returnedAt = null;
      }
    }

    await manager.save(Book, book);
  });

  loan.borrowerName = dto.borrowerName;
  loan.borrowerEmail = dto.borrowerEmail;
  loan.dueAt = newDueAt;
  loan.status = dto.status;

  loan.fine = loan.returnedAt
    ? LoanService.calculateFine(loan.dueAt, loan.returnedAt, this.finePerDay())
    : 0;

  return manager.save(Loan, loan);
});
}

```

Keterangan:

- `stockEffect()` — method **statik** yang memutuskan pengaruh perpindahan status terhadap stok buku. Karena sifatnya murni, logikanya dapat diuji secara unit tanpa menyentuh basis data.

- **Dipinjam → Dikembalikan** menambah stok dan mengisi tanggal kembali; **Dikembalikan → Dipinjam** mengurangi stok dan mengosongkannya kembali.
- Denda selalu **dihitung ulang**, sebab memajukan atau memundurkan tanggal jatuh tempo semestinya mengubah besaran dendanya juga.
- Bila stok tidak mencukupi, seluruh perubahan dibatalkan (rollback).

9.6 UPDATE — Pengembalian Buku dan Perhitungan Denda

```
// src/loan/loan.service.ts

/** Method STATIS – fungsi murni penghitung denda keterlambatan. */
static calculateFine(
  dueAt: Date,
  returnedAt: Date,
  ratePerDay: number,
): number {
  const lateDays = diffInDays(dueAt, returnedAt);

  if (lateDays <= 0) {
    return 0;
  }
  return lateDays * ratePerDay;
}

async returnBook(id: number): Promise<Loan> {
  return this.dataSource.transaction(async (manager) => {
    const loan = await manager.findOne(Loan, {
      where: { id },
      relations: { book: true },
    });

    if (!loan.isOngoing()) {
      throw new BadRequestException(
        `Transaksi ${loan.code} sudah dikembalikan sebelumnya.`
      );
    }

    const returnedAt = new Date();
    loan.returnedAt = returnedAt;
    loan.status = LoanStatus.RETURNED;
    loan.fine = LoanService.calculateFine(
      loan.dueAt,
      returnedAt,
      this.finePerDay(),
    );

    const book = await manager.findOne(Book, { where: { id: loan.bookId } });
    if (book) {
      book.increaseStock();
      await manager.save(Book, book);
    }

    return manager.save(Loan, loan);
  });
}
```

Keterangan:

- **isOngoing()** — method domain entitas yang mencegah pengembalian ganda.
- Denda dihitung otomatis: jumlah hari keterlambatan dikalikan tarif per hari yang dibaca dari variabel lingkungan **DENDA_PER_HARI**.

9.7 DELETE — Menghapus Data Buku dengan Penjagaan Integritas

```
// src/book/book.service.ts
async remove(id: number): Promise<void> {
  const book = await this.bookRepository.findOne({
    where: { id },
    relations: { loans: true },
  });

  if (!book) {
    throw new NotFoundException(`Buku dengan ID ${id} tidak ditemukan.`);
  }

  if (book.borrowedCount() > 0) {
    throw new BadRequestException(
      `Buku "${book.title}" tidak dapat dihapus karena sedang dipinjam.`
    );
  }

  const historyCount = book.loans?.length ?? 0;
  if (historyCount > 0) {
    throw new BadRequestException(
      `Buku "${book.title}" tidak dapat dihapus karena memiliki ` +
      `${historyCount} riwayat transaksi peminjaman. ` +
      `Riwayat transaksi wajib dipertahankan sebagai arsip perpustakaan.`
    );
  }

  await this.bookRepository.remove(book);
}
```

Keterangan:

- Pemeriksaan dilakukan **dua lapis**: buku yang sedang dipinjam, dan buku yang memiliki riwayat transaksi meski transaksinya sudah selesai.
- Lapis kedua diperlukan karena kolom `book_id` memakai batasan `ON DELETE RESTRICT`. Tanpa pemeriksaan ini, penghapusan ditolak langsung oleh PostgreSQL dan pengguna menerima pesan teknis berbahasa Inggris.
- `remove()` — menjalankan perintah `DELETE` pada basis data.

9.8 LIBRARY — pdfmake (Cetak Laporan PDF)


```
// src/report/pdf.service.ts
const PdfPrinter = require('pdfmake/src/printer');

@Injectable()
export class PdfService {
  private readonly printer = new PdfPrinter(STANDARD_FONTS);

  async buildLoanReport(
    loans: Loan[],
    summary: ReportSummary,
    from: Date,
    to: Date,
  ): Promise<Buffer> {
    const definition = this.buildDocDefinition(loans, summary, from, to);
    return this.render(definition);
  }

  /** PDFMake bekerja dengan aliran data, jadi potongannya digabung dahulu. */
  private render(definition: TDocumentDefinitions): Promise<Buffer> {
    return new Promise<Buffer>((resolve, reject) => {
      const doc = this.printer.createPdfKitDocument(definition);
      const chunks: Buffer[] = [];

      doc.on('data', (chunk: Buffer) => chunks.push(chunk));
      doc.on('end', () => resolve(Buffer.concat(chunks)));
      doc.on('error', (error: Error) => reject(error));

      doc.end();
    });
  }
}
```

```
// src/report/report.controller.ts
@Get('pdf')
async downloadPdf(
  @Res() res: Response,
  @Query('from') from?: string,
  @Query('to') to?: string,
) {
  const { startDate, endDate } = this.resolvePeriod(from, to);
  const loans = await this.loanService.findByPeriod(startDate, endDate);
  const file = await this.pdfService.buildLoanReport(
    loans, this.summarize(loans), startDate, endDate,
  );

  res.set({
    'Content-Type': 'application/pdf',
    'Content-Disposition': `attachment; filename="laporan-peminjaman.pdf"`,
    'Content-Length': file.length,
  });
  return res.end(file);
}
```

Keterangan:

- **PdfService membungkus** library pdfmake, sehingga controller tidak perlu mengetahui detail penyusunan dokumen (penerapan abstraksi).
- Memakai font bawaan standar PDF (Helvetica), sehingga tidak perlu menyertakan berkas **.ttf** — image Docker tetap ramping dan tidak mungkin gagal cetak.
- Header **Content-Disposition: attachment** membuat peramban langsung menawarkan kotak dialog penyimpanan berkas.

9.9 LIBRARY — Nodemailer (Pengiriman Surel)

```
// src/mail/mail.service.ts
import * as nodemailer from 'nodemailer';

@Injectable()
export class MailService implements OnModuleInit {
  private transporter: Transporter;

  onModuleInit(): void {
    this.transporter = nodemailer.createTransport({
      host: this.configService.get<string>('MAIL_HOST') ?? 'localhost',
      port: Number(this.configService.get<string>('MAIL_PORT') ?? 1025),
      secure: false,
      ignoreTLS: true,
    });
  }

  private async send(
    to: string,
    subject: string,
    html: string,
  ): Promise<boolean> {
    try {
      await this.transporter.sendMail({
        from: this.configService.get<string>('MAIL_FROM'),
        to,
        subject,
        html,
      });
      this.logger.log(`Surel terkirim ke ${to}: ${subject}`);
      return true;
    } catch (error) {
      this.logger.error(`Gagal mengirim surel ke ${to}`, errorStack(error));
      return false;
    }
  }
}
```

Keterangan:

- **OnModuleInit** — interface milik NestJS; koneksi SMTP dibangun **satukali** saat aplikasi dinyalakan, bukan setiap kali surel dikirim.
- Kegagalan pengiriman sengaja **ditangkap dan hanya dicatat ke log**, tidak dilempar ulang, supaya masalah pada server surel tidak ikut menggagalkan transaksi pinjaman yang datanya sudah tersimpan.
- Kredensial dibaca dari variabel lingkungan, tidak ditulis di dalam kode.

9.10 LIBRARY — class-validator dan class-transformer (Validasi)

```
// src/loan/dto/update-loan.dto.ts
export class UpdateLoanDto {
  @NotEmpty({ message: 'Nama peminjam wajib diisi.' })
  @IsString()
  @MaxLength(100, { message: 'Nama peminjam maksimal 100 karakter.' })
  borrowerName: string;

  @IsEmail({}, { message: 'Format email peminjam tidak valid.' })
  borrowerEmail: string;

  @IsISO8601({}, { message: 'Tanggal jatuh tempo tidak valid.' })
  dueAt: string;

  @IsEnum(LoanStatus, { message: 'Status pinjaman tidak dikenali.' })
  status: LoanStatus;
}
```

```
// src/common/validation.ts
export async function validateDto<T extends object>(
  cls: new () => T,
  data: Record<string, unknown>,
): Promise<ValidationResult<T>> {
  const dto = plainToInstance(cls, data);
  const failures = await validate(dto, { whitelist: true });

  const errors: string[] = [];
  for (const failure of failures) {
    for (const message of Object.values(failure.constraints ?? {})) {
      errors.push(message);
    }
  }

  return { dto, errors };
}
```

Keterangan:

- Aturan validasi ditempelkan langsung pada atribut dalam bentuk **decorator**, sehingga aturan bisnis menyatu dengan struktur datanya (enkapsulasi).
- Nama atribut memakai Bahasa Inggris, sedangkan PESAN errornya tetap Bahasa Indonesia karena pesan itulah yang dibaca pengguna.
- **plainToInstance** — mengubah data form HTML (yang seluruh nilainya bertipe string) menjadi instance kelas DTO dengan tipe data yang benar.
- Validasi dijalankan **di sisi server**, sehingga data tetap aman walaupun validasi bawaan peramban dilewati pengguna.

9.11 Penyiapan Data untuk Templat

```
// src/loan/loan.entity.ts
toView(reference: Date = new Date()) {
  return {
    ...this,
    displayStatus: this.displayStatus(reference),
    isOngoing: this.isOngoing(),
    overdueDays: this.overdueDays(reference),
    isOverdue: this.isOverdue(reference),
  };
}
```

Keterangan:

- Mesin templat Handlebars tidak dapat diandalkan untuk memanggil method sebuah kelas. `{{loan.displayStatus}}` memang dipanggil, tetapi `{{statusColor loan.displayStatus}}` justru menerima FUNGSI-nya, dan `{{#if loan.isOngoing}}` diblokir sama sekali oleh Handlebars 4.6+ karena method berada di prototype, bukan properti objek.
- `toView()` mengubah hasil method domain menjadi properti biasa, sehingga seluruh bentuk pemakaian di templat berperilaku sama dan dapat diprediksi.
- Seluruh controller memetakan entitas melalui method ini sebelum mengirimkannya ke templat.

10. Source Code Lengkap

Seluruh berkas source code sebagaimana tercantum pada tabel bagian **3.d** dipublikasikan pada repository berikut, dan dapat dijalankan kembali hanya dengan perintah `docker compose up --build`:

Repository: <https://github.com/irfanadwifangga/sertifikasi-lspedi>